Module 17 — Platform Engineering, Multi-Tenancy and Incident Response

> **Goal:** Run LLM capability as an internal product: paved roads for product teams, tenant isolation and fair scheduling, a correct and secure gateway, a GPU platform on Kubernetes that scales on the right signals, graceful degradation and disaster recovery, and an incident process that fits LLM-specific failures such as silent quality regressions and cross-tenant exposure.

Table of Contents

- 17.1 The LLM Platform as a Product: Paved Roads, Self-Service and Team Topology
- 17.2 Multi-Tenancy: Isolation, Fairness and Noisy Neighbours
- 17.3 LLM Gateway Design: Routing, Policy, Keys and Budgets
- 17.4 Kubernetes GPU Platform: Scheduling, Autoscaling and Upgrades
- 17.5 Reliability Engineering: Graceful Degradation, Failover and Disaster Recovery
- 17.6 Incident Response and Blameless Postmortems for LLM Systems

17.1 The LLM Platform as a Product: Paved Roads, Self-Service and Team Topology

Conceptual Deep Dive

Treat the platform team as a product team whose customers are product teams. The core artefact is a **paved road**: a template repository with tracing, an eval harness, guardrail hooks, the gateway client, a CI pipeline with the release gates, and a deployment manifest all pre-wired. Teams may leave the road, but then they own the cost: their own security review, on-call and evals. The road must be better than do-it-yourself, or nobody takes it.

Self-service means onboarding is a pull request or a form that provisions keys, quotas, model-alias access, dashboards and alerts automatically. Documentation, runnable examples and a changelog are part of the product, not an afterthought. Define platform **SLOs** (gateway availability, onboarding time, support response) and a support model (office hours, a request queue, an on-call for the shared plane). Platform APIs are versioned with a deprecation policy (announce, dual-run, remove) because provider churn forces frequent changes underneath and consumers must not feel every one.

Use the **Team Topologies** vocabulary (Skelton and Pais): stream-aligned teams own user-facing products; the platform team provides internal services; enabling teams coach adoption (evals, prompt practice); complicated-subsystem teams hold rare expertise such as inference-engine tuning. This clarifies who owns what and avoids a central team that does everything.

Measure the platform with outcome metrics: time to first successful call for a new team, adoption share of paved-road services, DORA-style lead time and change-failure rate, MTTR, cost-per-token trend and a periodic satisfaction survey. Platform output metrics (features shipped) mislead.

Start with the thinnest viable platform: gateway, tracing defaults and an eval gate; add capabilities when at least two teams ask for them. Resist becoming a gatekeeper; link to the maturity model in 1.6 and the build-versus-adopt framing in 1.7.

Edge cases. Teams with legitimate special needs (data residency, custom GPUs, a regulated workload) should get **extension points** (pluggable guardrail, custom endpoint registration, node-pool selectors) rather than forks that drift. During provider churn there is a velocity-versus-stability tension: pin platform contracts, absorb provider changes behind the gateway alias, and publish a compatibility matrix with dates.

Cross-Connection Scenarios

- The gateway (17.3) and tracing defaults (Module 10) are the first components of a paved road.
- Release gates (Module 13) and the evidence they emit are what the template wires in; governance tiers (16.1) select which gates are mandatory.
- Cost attribution labels (11.8) depend on onboarding setting an owner and cost centre.
- The maturity levels in 1.6 tell you which platform capability to build next.

Production Anti-Patterns

- Mandating the platform before it is better than doing it yourself, which breeds workarounds and shadow stacks.
- No docs or runnable examples, so every onboarding becomes a support ticket and a meeting.
- Measuring the platform team on shipped features instead of customer outcomes such as time to first call and change-failure rate.

Minimal Working Code

An onboarding scaffolder generates a service config with required controls, and a policy-as-code validator rejects incomplete ones.

```
REQUIRED = {"owner": str, "tracing": bool, "e
val_gate": bool, "monthly_token_quota": int, "cost_centre": str}

def scaffold(name: str, owner: str, cost_centre: str, tier: int = 2) -> dict:
    return {"name": name, "owner": owner, "cost_centre": cost_centre, "
;tier": tier,
            "tracing": True, "eval_gate": True, "monthly_token_quota": 50_0
00_000 if tier <= 2 else 5_000_000,
            "guardrails": ["pii_redact", "injection_scan"]}

def validate(cfg: dict) -> list[str]:
    errs = [f"missing/invalid {k}" for k, t in REQUIRED.items() if not isinstance(cfg.get(k), t
) or cfg.get(k) in ("", None)]
    if cfg.get("tier", 1) >= 3 and "human_review" not in cfg.get("guardrails&
quot;, []):
        errs.append("tier>=3 needs human_review guardrail")
    if cfg.get("tracing") is False:
        errs.append("tracing may not be disabled")
    return errs

good = scaffold("support-chat", "team-support", "cc-114")
bad = dict(good, tracing=False, tier=3)
print(validate(good), validate(bad))
assert validate(good) == [] and len(validate(bad)) == 2
```

A platform scorecard computes delivery and adoption metrics from event records.

```
from statistics import median

def scorecard(deploys: list[dict], incidents: list[dict], services: list[dict]) -> dict:
    lead = median(d["merged_h"] for d in deploys)                # hours from merge to prod
    cfr = sum(d["failed"] for d in deploys) / len(deploys)       # change-failure rate
    mttr = sum(i["resolved_min"] for i in incidents) / len(incidents)
    adoption = sum(s["paved_road"] for s in services) / len(services)
```

```
    return {"lead_time_h": lead, "change_failure_rate": round(cfr, 2),
            "mttr_min": round(mttr, 1), "adoption": round(adoption, 2)}

deploys = [{"merged_h": 2, "failed": False}, {"merged_h": 6, "failed&q
uot;: True},
           {"merged_h": 3, "failed": False}, {"merged_h": 4, "failed&q
uot;: False}]
incidents = [{"resolved_min": 35}, {"resolved_min": 80}]
services = [{"paved_road": True}, {"paved_road": True}, {"paved_road": Fals
e}]
print(scorecard(deploys, incidents, services))
```

Interview Angle

Q: How would you design the internal platform for 20 product teams adopting LLMs? **A:** Thin first: a gateway with virtual keys and budgets, tracing defaults, a template repo with an eval gate, and self-service onboarding; then add shared retrieval and guardrail services as demand appears. Trade-off: shared services speed teams but couple them to platform velocity; offer extension points. Check: time to first successful call under a day, and paved-road adoption rising without a mandate.

Q: What do you standardise and what do you leave to teams? **A:** Standardise cross-cutting, high-risk or high-cost concerns: authentication, tracing schema, cost attribution, eval gating, guardrail baseline, release mechanism. Leave prompts, task design, domain evals and model choice (within an approved list). Trade-off: more standards reduce variance but slow experimentation. Check: audit shows every service emits the common trace schema and has a quota, with no platform tickets needed for routine changes.

Q: How do you know the platform is succeeding? **A:** Outcome metrics: time to first call, adoption, change-failure rate, MTTR, cost-per-token trend and team satisfaction, trended per quarter. Trade-off: surveys are noisy and adoption can be forced; pair them with voluntary-use data. Check: teams that left the paved road return, or stay for stated reasons that feed the roadmap.

17.2 Multi-Tenancy: Isolation, Fairness and Noisy Neighbours

Conceptual Deep Dive

Multi-tenancy is a spectrum: shared everything; per-tenant namespace, index and cache; per-tenant adapter on shared base weights; dedicated replicas; dedicated cluster. Cost and operational burden rise with isolation, so choose per tenant tier and data sensitivity: a free tier shares everything with logical separation, a regulated enterprise may get dedicated replicas and keys.

Data isolation has several surfaces. For retrieval, choose between ACL filters on a shared collection and separate collections or partitions (Module 06 section 3.9 covers partition keys); filters are cheaper but a missed filter is a leak, so test them continuously. Cache keys must include tenant (and policy version); legacy Module 11 incident 1 describes a cross-tenant cache problem. Segregate logs and traces, use per-tenant encryption keys where contracts demand it, and apply prefix-cache and KV-reuse boundaries so one tenant's prompt prefix cannot be observed by another through timing.

Performance isolation protects everyone from the noisy neighbour: quotas on requests and tokens, priority classes, fair scheduling such as weighted fair queueing or **deficit round robin** across tenant queues, admission control at the gateway, and per-tenant caps on concurrent sequences and KV-cache share. Without these, one tenant's batch job fills the queue and every other tenant's TTFT collapses.

Per-tenant configuration (prompts, guardrails, model alias, retention) needs schema validation and review, because a bad tenant config is a production change. Tie **SLO tiers** to billing: premium tenants get higher priority and larger quotas. Detect noisy neighbours from per-tenant metrics, but bound cardinality: report top-N tenants by consumption and keep full per-tenant detail in logs or sampled traces rather than as metric labels.

Offboarding must delete tenant data across every store (index, cache, logs, adapters, backups on schedule) and produce a deletion certificate (8.7). Run tenant-specific evals when the tenant has custom prompts or data.

Edge cases. One tenant's burst can trigger preemptions that restart other tenants' sequences under KV pressure; cap per-tenant concurrency below the point where eviction starts. Cross-tenant leakage via shared caches or prefix reuse is subtle: semantic caches match by similarity, so a near-duplicate query from another tenant can return the first tenant's answer.

Cross-Connection Scenarios

- Admission control, queues and KV pressure (Module 05, 5.8) are where engine-level fairness is enforced.
- Gateway quotas and virtual keys (17.3) are the first line of tenant control.
- Tenant partition design and ACLs (Module 06) determine retrieval isolation.
- Governance tiers (16.1) and cross-tenant exposure as a top-severity incident (17.6).

Production Anti-Patterns

- Isolation only by a WHERE clause nobody tests, so the first missed filter becomes a data breach.
- FIFO queues shared across tenants, so one batch job sets everyone's latency.
- Tenant id missing from cache keys, so the cache silently serves another tenant's answer.

Minimal Working Code

A deficit-round-robin simulation compares per-tenant waiting time against FIFO when one tenant bursts.

```
from collections import deque

def simulate(policy: str, ticks: int = 80, budget: int = 300, cost: int = 100) -> dict:
    qs = {t: deque() for t in ("batch", "alice", "bob")}
    fifo, deficit, waits = deque(), {t: 0 for t in qs}, {t: [] for t in qs}
    for tick in range(ticks):
        arrivals = (["batch"] * 60 if tick == 0 else []) + (["alice", "bob"
] if tick < 20 else [])
        for t in arrivals:
            qs[t].append(tick); fifo.append((t, tick))
        left = budget
        if policy == "fifo":
            while fifo and left >= cost:
                t, a = fifo.popleft(); qs[t].popleft(); waits[t].append(tick - a); left -= cost
        else:
            while left >= cost and any(qs.values()):
                for t in qs:
                    if not qs[t]:
                        deficit[t] = 0; continue
                    deficit[t] += cost
                    while qs[t] and deficit[t] >= cost and left >= cost:
                        waits[t].append(tick - qs[t].popleft()); deficit[t] -= cost; left -= cost
    return {t: round(sum(w) / len(w), 1) for t, w in waits.items() if w}

print("fifo", simulate("fifo"))
print("drr ", simulate("drr"))
assert simulate("drr")["alice"] < simulate("fifo")["alice"]
```

A cross-tenant leak tester plants unique canaries and asserts none cross tenants, catching a cache keyed without tenant.

```
class Stack:
    def __init__(self, tenant_in_cache_key: bool):
        self.docs, self.cache, self.keyed = {}, {}, tenant_in_cache_key

    def ingest(self, tenant: str, text: str):
        self.docs.setdefault(tenant, []).append(text)

    def answer(self, tenant: str, query: str) -> str:
        key = (tenant, query) if self.keyed else (query,)
        if key not in self.cache:
            self.cache[key] = " ".join(d for d in self.docs.get(tenant, []) if query in d) or &
quot;no result"
        return self.cache[key]

def leak_test(stack: Stack, tenants: list[str]) -> list[str]:
    canary = {t: f"CANARY-{t}-9f3a" for t in tenants}
    for t in tenants:
        stack.ingest(t, f"shared-topic {canary[t]}")
    leaks = []
```

```
    for t in tenants:                    # every tenant asks the same query
        out = stack.answer(t, "shared-topic")
        leaks += [f"{t} saw {c}" for o, c in canary.items() if o != t and c in out]
    return leaks

print("safe :", leak_test(Stack(True), ["acme", "globex"]))
print("buggy:", leak_test(Stack(False), ["acme", "globex"]))
assert not leak_test(Stack(True), ["a", "b", "c"]) and leak_test(Stack(Fals
e), ["a", "b"])
```

Interview Angle

Q: Design isolation for 300 enterprise tenants with different data sensitivity. **A:** Tier it: most tenants share infrastructure with namespaced indexes, tenant-keyed caches and per-tenant keys in a KMS; sensitive tenants get dedicated partitions or replicas, and the few regulated ones dedicated clusters. Trade-off: each step up multiplies cost and operations. Check: continuous canary leak tests per tenant pair and an access-log audit; zero canary appearances, with the test also failing on a seeded bug.

Q: One tenant's batch job slows everyone. What controls exist at gateway, scheduler and engine levels? **A:** Gateway: per-tenant token and request quotas, priority classes, admission control. Scheduler: weighted fair queueing or DRR, separate pools for batch. Engine: per-tenant concurrency caps and KV budget to avoid preemption. Trade-off: strict caps waste idle capacity; allow borrowing with reclaim. Check: other tenants' p95 TTFT stays within SLO during an injected burst test.

Q: How do you test for cross-tenant leakage continuously? **A:** Plant unique canary strings per tenant in each store (documents, caches, memory), query from other tenants including near-duplicates, and alert on any appearance; run in CI and in production synthetic probes. Trade-off: canaries only find what they exercise; add fuzzed queries and code review of key construction. Check: probe pass rate and a deliberately injected leak that must trigger the alert.

17.3 LLM Gateway Design: Routing, Policy, Keys and Budgets

Conceptual Deep Dive

The gateway is the single enforcement point between clients and model providers or self-hosted engines. Its responsibilities: authentication, **virtual keys** (per-team or per-app keys mapped to real provider credentials), budgets, rate limits, request normalisation to one API surface (usually OpenAI-compatible), model-alias routing, retries and fallbacks, caching, guardrail hooks, logging and tracing, and cost metering. Open-source and commercial options exist (LiteLLM, Envoy-based AI gateways, cloud provider gateways); decide build versus adopt using 1.7, and note that adopting still leaves you owning configuration, upgrades and the hot path.

Streaming correctness is the hardest part. Pass server-sent events through without buffering (buffering destroys TTFT), apply backpressure so slow clients do not balloon memory, cancel the upstream request when the client disconnects (otherwise you pay for tokens nobody reads and hold a GPU slot), and account usage correctly when a stream ends abnormally by estimating from tokens seen or reading the final usage chunk where available.

Alias indirection: clients call a stable name such as `support-chat`; configuration maps it to a deployment or bundle (13.3). Canaries, A/B tests, fallbacks and rollbacks then need no client change, and the alias registry becomes the unit of release.

Reliability: per-hop timeouts, retry budgets (cap retries as a fraction of traffic), circuit breakers per upstream, idempotency keys for retried requests, and high availability with shared counters in Redis or similar. Set an overhead budget (for example single-digit milliseconds at p99 added latency, a target to measure, not a guarantee).

Security: store provider keys in a secrets manager and rotate them; issue virtual keys that can be revoked individually and restricted to models; defend against SSRF if clients may supply base URLs (allow-list); strip or validate headers to prevent injection; log bodies only by explicit opt-in with redaction. Keep configuration as reviewed code, since a routing change is a production change.

Edge cases. A **mid-stream upstream failure** leaves a partial response: decide semantics (terminate with an error event, never silently retry a streamed answer the client has partly shown; retry only before the first token). **Clock skew** and distributed counters make rate limits approximate; use a single logical clock source (Redis time), accept slight over-admission, and prefer token buckets with atomic scripts.

Cross-Connection Scenarios

- Alias-based canaries and rollbacks are the release mechanism in 13.3.
- Per-tenant quotas and fair queueing (17.2) are enforced here; guardrail hooks (Module 08) run in the pipeline.
- Cost metering feeds attribution (11.8); traces feed Module 10 dashboards.
- Failover and degradation (17.5) are configured as routing policy.

Production Anti-Patterns

- A gateway that buffers full responses before sending, destroying TTFT and hiding streaming bugs.
- Retrying non-idempotent tool calls or requests whose side effects already happened, producing duplicates.
- Logging request and response bodies by default, creating an unreviewed personal-data store.

Minimal Working Code

A middleware pipeline with fake upstreams and a test for each rejection path.

```
import asyncio

class Reject(Exception):
    def __init__(self, status): self.status = status

KEYS = {"k-team-a": {"budget": 1000, "spent": 0, "models": {"
;support-chat"}}}
ALIASES = {"support-chat": "upstream-small"}
UPSTREAMS = {"upstream-small": lambda p: f"echo:{p}"}
hits = {}

async def handle(key: str, alias: str, prompt: str, rate_limit: int = 3) -> tuple[int, str]:
    try:
        rec = KEYS.get(key)
        if rec is None: raise Reject(401)                       # auth
        if rec["spent"] >= rec["budget"]: raise Reject(402)             # budget
        hits[key] = hits.get(key, 0) + 1
        if hits[key] > rate_limit: raise Reject(429)                    # rate limit
        if alias not in rec["models"] or alias not in ALIASES: raise Reject(404)   # route
        out = UPSTREAMS[ALIASES[alias]](prompt)                         # upstream
        rec["spent"] += len(prompt) + len(out)
        return 200, out
    except Reject as r:
        return r.status, "rejected"

async def main():
    assert (await handle("bad", "support-chat", "hi"))[0] == 401
    assert (await handle("k-team-a", "unknown", "hi"))[0] == 404
    assert (await handle("k-team-a", "support-chat", "hi"))[0] == 200
    KEYS["k-team-a"]["spent"] = 1000
    assert (await handle("k-team-a", "support-chat", "hi"))[0] == 402
    print("all rejection paths covered")
asyncio.run(main())
```

A virtual-key manager with soft and hard limits, atomic spend under threads and rotation.

```
import secrets, threading

class KeyManager:
    def __init__(self):
        self.lock, self.keys, self.by_secret = threading.Lock(), {}, {}

    def create(self, name: str, hard: int, soft_ratio: float = 0.8) -> str:
        s = "vk-" + secrets.token_hex(8)
        self.keys[name] = {"hard": hard, "soft": int(hard * soft_ratio), "spent&
quot;: 0, "warned": False}
        self.by_secret[s] = name
        return s
```

```
    def rotate(self, old_secret: str) -> str:
        with self.lock:
            name = self.by_secret.pop(old_secret)
            new = "vk-" + secrets.token_hex(8)
            self.by_secret[new] = name
            return new

    def spend(self, secret: str, cost: int) -> str:
        with self.lock:                          # check-and-add must be atomic
            k = self.keys[self.by_secret[secret]]
            if k["spent"] + cost > k["hard"]:
                return "denied"
            k["spent"] += cost
            return "warn" if k["spent"] >= k["soft"] else "ok"

km = KeyManager()
s = km.create("team-a", hard=500)
results = []
ts = [threading.Thread(target=lambda: [results.append(km.spend(s, 10)) for _ in range(10)]) for _ in rang
e(8)]
[t.start() for t in ts]; [t.join() for t in ts]
print(results.count("denied"), km.keys["team-a"]["spent"])
assert km.keys["team-a"]["spent"] == 500 and results.count("denied") == 30
```

Interview Angle

Q: Design the gateway for an internal LLM platform. What are the hard parts? **A:** Streaming pass-through with correct cancellation and usage accounting, distributed rate limits and budgets, alias-based routing with safe fallbacks, and security of keys and logs. Trade-off: richer features add latency and a single point of failure; keep the hot path thin and run HA with a per-request overhead budget. Check: added p99 latency, TTFT delta versus direct calls, and chaos tests killing upstream mid-stream.

Q: How do you canary a new model behind the gateway? **A:** Create a new bundle, point a weighted share of the alias at it (1 to 5 percent), pin users by hash for consistency, and compare quality and latency to control with a pre-declared analysis and auto-rollback conditions. Trade-off: small canaries detect only large effects; combine with shadow traffic. Check: rollback by alias flip completes under the rehearsed time with no client change.

Q: A client disconnects mid-stream. What should the gateway do and why? **A:** Detect the disconnect, cancel the upstream request to free the GPU slot and stop token spend, record partial usage with a "cancelled" status, and avoid retrying. Trade-off: cancellation can cut off a response meant for logging or caching; decide per route. Check: after disconnect tests, upstream active-sequence count drops within a second or two and billed tokens match emitted tokens.

17.4 Kubernetes GPU Platform: Scheduling, Autoscaling and Upgrades

Conceptual Deep Dive

GPU scheduling basics. The NVIDIA device plugin (usually installed through the GPU Operator) exposes GPUs as schedulable resources. Dedicate node pools with taints and tolerations so non-GPU workloads stay off expensive nodes; set explicit resource requests and limits. Topology matters: place tensor-parallel replicas on the same node within one NVLink domain, because inter-node tensor parallelism over ordinary networking is slow. Multi-node jobs (training, multi-node inference) need **gang scheduling** so all pods start together or not at all; Kueue and Volcano are common examples. Use priorities and preemption deliberately (online serving above batch), and MIG slices to carve a GPU for small models where hardware supports it.

Weight loading dominates cold start. Options: bake weights into the image (fast pull on warm nodes, huge images), load from object storage with a local NVMe cache, or mount a shared volume. Cold-start time sets your scaling policy: with a four-minute start you cannot react to a spike, so keep headroom. Define readiness as model loaded, warm-up request completed and compiled graphs captured, not process started.

Autoscaling signals. CPU and GPU utilisation mislead because decoding saturates memory bandwidth and KV cache before utilisation looks high. Scale on queue depth, running and waiting requests, KV-cache usage and TTFT/ITL SLO burn (Module 05 section 3.4); metric names vary by engine, so check current docs. Add hysteresis and cooldowns to stop flapping, and consider scale-to-zero only for tolerable cold starts.

Graceful termination. On SIGTERM stop accepting new requests, let streaming connections drain, use a PodDisruptionBudget so voluntary evictions cannot remove too many replicas, and set terminationGracePeriodSeconds above the longest expected stream (legacy Module 05 Incident 2).

Upgrades. Driver, CUDA and engine versions must match. Use blue-green or canary node pools, pin image digests, upgrade a small pool, run a smoke and eval suite, then widen. Observability and cost: DCGM exporter metrics, labels for cost attribution (11.8).

Edge cases. A node-pool scale-up can take ten minutes or more (instance provisioning, driver install, image pull): hold warm capacity and use predictive or scheduled scaling for known peaks. A driver mismatch between the node and the container's CUDA runtime fails at startup or silently degrades; check compatibility in CI against the node image.

Cross-Connection Scenarios

- Engine metrics and the knee of the throughput curve (Module 05, 5.8) provide the autoscaling targets.
- Warm capacity and cost trade-offs connect to Module 11 and cost attribution in 11.8.
- Progressive delivery (13.3) applies to node pools as well as models.
- Capacity planning for projects in 18.3 uses per-replica goodput measured here.

Production Anti-Patterns

- Autoscaling on CPU, which stays low while the KV cache saturates and requests queue.
- A readiness probe that returns OK before weights are loaded and warmed, sending traffic to a pod that times out.
- In-place driver upgrades on all nodes at once, with no canary pool and no rollback path.

Minimal Working Code

An autoscaler simulation compares hysteresis plus cooldown with a naive threshold policy under bursty traffic and a cold start.

```
import random

def run(policy: str, minutes: int = 120, cap: int = 20, cold: int = 4, seed: int = 3) -> dict:
    rng = random.Random(seed)
    ready, pending, queue, last_change = 2, [], 0, -99
    events = breach = 0
    for m in range(minutes):
        load = 30 + (80 if 30 <= m < 50 else 0) + rng.randint(-5, 5) + (60 if 80 <= m < 85 el
se 0)
        ready += sum(1 for t in pending if t <= m); pending = [t for t in pending if t > m]
        queue = max(0, queue + load - ready * cap)
        breach += queue > 40
        total = ready + len(pending)
        if queue > 40 and total < 12 and (policy == "naive" or not pending):
            pending += [m + cold] * (1 if policy == "naive" else 3); events += 1; last_change =
 m
        elif queue == 0 and load < ready * cap * 0.5 and ready > 2:
            if policy == "naive" or m - last_change >= 10:
                ready -= 1; events += 1; last_change = m
    return {"scale_events": events, "slo_breach_min": breach}

print("naive", run("naive")); print("hyst ", run("hysteresis"))
assert run("hysteresis")["scale_events"] < run("naive")["scale_even
ts"] and run("hysteresis")["slo_breach_min"] <= run("naive")["
slo_breach_min"]
```

A rolling-upgrade planner computes the timeline and capacity dip from surge, unavailability, warm-up and a PDB.

```
import math

def plan_rollout(replicas: int, max_unavailable: int, max_surge: int, warmup_min: int, pdb_min_available:
 int) -> dict:
    if max_surge > 0:
        wave, dip = max_surge, 0                # new pods warm before old ones are removed
    else:
```

```
        wave = min(max_unavailable, replicas - pdb_min_available)
        if wave < 1:
            raise ValueError("PDB forbids any disruption: raise replicas or lower minAvailable"
)
        dip = wave
    waves = math.ceil(replicas / wave)
    return {"waves": waves, "total_min": waves * warmup_min, "min_ready": r
eplicas - dip,
            "capacity_dip_pct": round(100 * dip / replicas, 1)}

print(plan_rollout(replicas=8, max_unavailable=1, max_surge=0, warmup_min=6, pdb_min_available=6))
print(plan_rollout(replicas=8, max_unavailable=0, max_surge=2, warmup_min=6, pdb_min_available=8))
try:
    plan_rollout(4, 1, 0, 6, 4)
except ValueError as e:
    print("blocked:", e)
```

Interview Angle

Q: Design autoscaling for a vLLM deployment with 4-minute cold starts. **A:** Scale on queue depth and KV-cache usage with TTFT SLO burn as a guard, keep minimum warm replicas sized to the burst you can absorb in four minutes, add scheduled scaling for known peaks, use hysteresis and cooldowns, and cache weights on local NVMe to shorten starts. Trade-off: headroom costs money. Check: replay a traffic burst in simulation and staging; SLO-breach minutes and flapping events within targets.

Q: How do you upgrade GPU drivers across a 200-node fleet safely? **A:** Create a canary node pool with the new driver and matching image, run smoke and eval tests plus a soak period, shift a small traffic share, then roll in waves under a PDB, keeping the old pool for rollback. Trade-off: duplicate capacity during the roll costs money. Check: error rate, latency and GPU error counters (DCGM) per wave, with a rehearsed rollback time.

Q: Why is CPU-based HPA wrong for LLM serving? **A:** Decoding is bound by memory bandwidth and KV capacity, so CPU stays low while requests queue, and GPU utilisation can read high at low useful throughput. The signal that predicts user pain is queue length, waiting requests and KV usage. Trade-off: custom metrics need an adapter such as KEDA or a metrics pipeline. Check: correlate candidate signals with p95 TTFT under load tests; choose the one that leads it.

17.5 Reliability Engineering: Graceful Degradation, Failover and Disaster Recovery

Conceptual Deep Dive

List **failure domains**: model provider, region, GPU pool, vector database, cache, guardrail service, tool APIs, authentication, and the gateway itself. For each, state what the user experiences when it fails. Reliability work is deciding that in advance.

Build a **degradation ladder** with explicit triggers: full service; then a smaller or cheaper model; then a cached or static answer; then retrieval-only results with citations; then a clear "temporarily unavailable" message. Each rung needs an owner-approved trigger (health checks, error rate, latency budget), a user-visible indicator where honest, and a test that exercises it. A rung never tested is a hypothesis.

Overload control protects the system from itself: timeouts at every hop, retry budgets (retries limited to a share of traffic with jitter), **load shedding** by priority and deadline (drop work whose deadline has already passed), backpressure, bulkheads (separate pools so batch cannot starve interactive), circuit breakers, and hedged requests used sparingly because they add load exactly when capacity is scarce.

Dependency arithmetic: serial components multiply availability (three 99.9 percent services in series give about 99.7 percent); parallel redundancy improves it only if failures are independent. Make optional dependencies truly optional: if the reranker fails, skip it and flag reduced quality. Guardrail failure needs a decision by risk tier: **fail-open** keeps service but removes protection; **fail-closed** keeps safety but loses availability. Low-risk internal tools may fail open; anything with tool access or regulated output fails closed or degrades to read-only.

Disaster recovery sets RTO (time to restore) and RPO (tolerable data loss) per component: vector indices rebuilt from source versus restored from snapshots, model weights in a registry, fine-tuning checkpoints, configuration and alias registry, audit logs. Choose between multi-region active-active (cost, consistency complexity) and standby (cheaper, slower), and run restore drills on a calendar. Practise with **game days** and chaos experiments, and prepare status communication templates.

Edge cases. A fail-open guardrail during an outage can create a security incident; the decision must be recorded per tier and the traffic flagged for later review. Failover can overload the secondary region; reserve capacity or apply admission control so degraded service is predictable instead of both regions collapsing.

Cross-Connection Scenarios

- Gateway routing and fallbacks (17.3) implement the ladder; autoscaling limits (17.4) bound recovery speed.
- Tier-dependent fail-open versus fail-closed choices come from governance tiers (16.1).
- Backups of indices and registries link to Module 06; audit-store restore tests link to 16.4.
- The RAG reliability budget in Module 12 section 2.4 gives the serial maths.

Production Anti-Patterns

- Untested fallback paths that fail the first time they are needed.
- Retry storms that multiply load on a recovering dependency and extend the outage.
- A single global vector database with no restore drill and no recorded RTO/RPO.

Minimal Working Code

A composite availability calculator and a degradation-ladder policy engine.

```
from math import prod

def serial(avails: list[float]) -> float:
    return prod(avails)

def parallel(avails: list[float]) -> float:
    return 1 - prod(1 - a for a in avails)

def choose_mode(h: dict) -> str:
    """h: booleans/floats from health checks. Returns the highest rung that is currently s
afe."""
    if not h["gateway_ok"]:
        return "unavailable"
    if h["guardrail_ok"] is False and h["tier"] >= 3:
        return "unavailable"                          # fail-closed for high tiers
    if h["primary_ok"] and h["error_rate"] < 0.02:
        return "full" if h["reranker_ok"] else "full_no_rerank"
    if h["fallback_ok"]:
        return "smaller_model"
    if h["cache_hit_rate"] > 0.3:
        return "cached"
    return "retrieval_only" if h["vector_ok"] else "unavailable"

print(round(serial([0.999, 0.999, 0.999]), 5), round(parallel([0.99, 0.99]), 5))
base = dict(gateway_ok=True, guardrail_ok=True, tier=2, primary_ok=False, error_rate=0.4,
            reranker_ok=True, fallback_ok=False, cache_hit_rate=0.1, vector_ok=True)
print(choose_mode(base), choose_mode({**base, "fallback_ok": True}), choose_mode({**base, "
;guardrail_ok": False, "tier": 3}))
assert choose_mode(base) == "retrieval_only"
```

A load shedder with priority queues, deadline-aware dropping and a token bucket, under an overload simulation.

```
import random
from collections import deque

def simulate(ticks=200, capacity=6, max_queue=30, seed=7):
    rng = random.Random(seed)
    qs = {0: deque(), 1: deque(), 2: deque()}             # 0 = interactive, 2 = batch
    bucket, served, shed = 10.0, {0: 0, 1: 0, 2: 0}, {0: 0, 1: 0, 2: 0}
    for t in range(ticks):
        bucket = min(10.0, bucket + 2.0)                  # batch admission refills slowly
        for _ in range(rng.randint(5, 14)):               # offered load exceeds capacity
            p = rng.choices((0, 1, 2), (0.3, 0.3, 0.4))[0]
```

```
            if p == 2 and bucket < 1:
                shed[p] += 1; continue
            bucket -= p == 2
            if sum(map(len, qs.values())) >= max_queue:
                low = max(k for k in qs if qs[k])          # evict lowest priority present
                if low > p: qs[low].popleft(); shed[low] += 1
                else: shed[p] += 1; continue
            qs[p].append(t + 5)                            # deadline = 5 ticks
        for k in qs:
            while qs[k] and qs[k][0] < t: qs[k].popleft(); shed[k] += 1
        for _ in range(capacity):
            k = next((k for k in qs if qs[k]), None)
            if k is not None: qs[k].popleft(); served[k] += 1
    return served, shed

served, shed = simulate()
print("served", served, "shed", shed)
assert served[0] > served[2] and shed[2] > shed[0]
```

Interview Angle

Q: Your reranker service is down. What does the system do and who decided that? **A:** It skips reranking, serves first-stage ranking with a "reduced quality" flag, alerts, and tightens context size; the decision was made in advance by the product owner and on-call engineer and recorded in the degradation ladder. Trade-off: quality drops for tail queries but availability holds. Check: a game-day test that kills the reranker and measures the offline quality delta and latency.

Q: Design DR for a RAG platform: RTO/RPO per component. **A:** Source documents and registries: RPO near zero via replication; vector indices: rebuildable, with snapshots to bound RTO; model weights: immutable registry copies in a second region; audit logs: durable with chain checks; cache: disposable. Trade-off: active-active halves risk but doubles cost. Check: a timed restore drill per component against stated RTO, repeated quarterly.

Q: How do you test graceful degradation? **A:** Fault injection per failure domain in staging and limited production, with expected mode assertions: kill the primary, block the vector DB, add latency, flip the guardrail unhealthy, and verify the ladder rung, user message and alerting. Trade-off: production chaos risks users; start in staging, then small blast radius. Check: every rung exercised in the last quarter with recorded results.

17.6 Incident Response and Blameless Postmortems for LLM Systems

Conceptual Deep Dive

LLM systems add incident types beyond ordinary outages: latency regression, **silent quality regression** (answers worsen with no errors), safety incidents (harmful or policy-violating output), data exposure (cross-tenant or training-data leakage), cost spikes or runaway agents, prompt injection and exfiltration, and provider-side incidents. Each needs a detection source and a first response.

Define a severity matrix with LLM-specific rules: any confirmed cross-tenant data exposure is top severity regardless of volume; a safety incident affecting vulnerable users outranks a latency issue; a runaway agent that takes irreversible actions is top severity. Roles: **incident commander** (decides, coordinates), ops lead (fixes), communications lead (stakeholders, status page), scribe (timeline). Keep an on-call rotation with clear escalation and handoff.

The **first 15 minutes** are about containment and evidence: stop the bleeding with a kill switch, alias rollback, disabling a tool or retrieval source, forcing the fallback model or tightening a guardrail; preserve evidence (traces, bundle ids, prompts under access control, tenant ids) before logs rotate; communicate status early with what is known and when the next update comes.

Diagnose by "what changed?": model or provider version, prompt, index or embedding, configuration and flags, traffic mix, dependency versions, upstream data. Release markers on dashboards and bundle ids in traces make this a query. Quality incidents have no hard error signal, so define detection thresholds in advance (10.7): judge-score drift, thumbs-down rate, escalation rate, refusal rate.

The **blameless postmortem** records timeline, impact, root cause and contributing factors, detection gap (how long before we knew, and why), what went well, and action items with an owner and due date. Add a regression test to the golden or red-team set so the failure cannot recur silently. Track MTTD (time to detect), **MTTR** (time to restore) and recurrence. Handle personal data in

incident documents carefully: reference trace ids and redact content. Blameless means analysing system conditions, not people; it still requires accountability for follow-through.

Edge cases. Provider-side incidents cannot be fixed by you, but you can degrade: fail over to a secondary provider, serve cached answers, or reduce features, with communication to customers. Without pre-declared quality thresholds, teams argue about whether a drop is real while it continues; the threshold and the noise floor from your eval harness settle that.

Cross-Connection Scenarios

- Release markers and alias rollback (13.3, 17.3) make rollback and "what changed" fast.
- Degradation modes (17.5) are the containment toolbox; tenant exposure (17.2) is the highest-severity example.
- Detection thresholds and drift metrics (10.7) feed alerts; audit trails (16.4) provide reconstruction evidence.
- Injection and exfiltration incidents link to Module 08; runaway agents to budgets in Module 09.

Production Anti-Patterns

- Debugging in production without recording what changed, so the fix cannot be tied to a cause.
- Postmortem action items without owners or dates, which are never completed.
- Blaming the model instead of the missing guardrail, test or alert that let the failure reach users.

Minimal Working Code

A "what changed" correlator ranks candidate causes by proximity to anomaly start and blast radius.

```
from datetime import datetime

def rank_causes(events: list[dict], anomaly_start: datetime, window_min: int = 120) -> list[tuple[floa
t, str]]:
    scored = []
    for e in events:
        lag = (anomaly_start - e["ts"]).total_seconds() / 60
        if lag < 0 or lag > window_min:          # after the anomaly, or too early
            continue
        score = e["blast_radius"] / (1 + lag / 10)     # nearer + wider scope = more suspicious
        scored.append((round(score, 3), f"{e['kind']}: {e['name']} ({lag:.0f} min before)"))
    return sorted(scored, reverse=True)

t = lambda hm: datetime(2026, 9, 1, *map(int, hm.split(":")))
events = [
    {"ts": t("01:10"), "kind": "deploy", "name": "
retriever v41", "blast_radius": 0.6},
    {"ts": t("01:50"), "kind": "config", "name": "
reranker top_k 20->5", "blast_radius": 0.8},
    {"ts": t("01:55"), "kind": "provider", "name": &quo
t;model alias re-pointed", "blast_radius": 1.0},
    {"ts": t("02:30"), "kind": "flag", "name": "ne
w UI banner", "blast_radius": 0.1},
]
for s, d in rank_causes(events, t("02:00")):
    print(s, d)
```

A postmortem generator validates the structured record, requiring owner, date and regression-test link on every action item.

```
def postmortem(rec: dict) -> str:
    errs = [f"action '{a['what']}' lacks {f}" for a in rec["actions"] for f in ("
;owner", "due", "regression_test") if not a.get(f)]
    if not rec.get("detection_gap_min"):
        errs.append("detection gap not recorded")
    if errs:
        raise ValueError("; ".join(errs))
    tl = "\n".join(f"- {ts}: {ev}" for ts, ev in rec["timeline"])
    ac = "\n".join(f"- [ ] {a['what']} (owner {a['owner']}, due {a['due']}, test {a['regre
ssion_test']})" for a in rec["actions"])
    return (f"# Postmortem: {rec['title']}\n\n**Severity:** {rec['severity']}  **MTTD:** {rec['detec
```

```
tion_gap_min']} min  "
          f"**MTTR:** {rec['mttr_min']} min\n\n## Timeline\n{tl}\n\n## Root cause and contributing
 factors\n{rec['root_cause']}\n\n## Actions\n{ac}\n")

rec = {"title": "Quality drop after alias re-point", "severity": "SEV2
", "detection_gap_min": 95, "mttr_min": 130,
      "timeline": [("02:00", "judge score drop alert"), ("02:20"
, "alias rolled back")],
      "root_cause": "Provider re-pointed alias; no dated model pin; weak canary alert.&qu
ot;,
      "actions": [{"what": "pin dated model versions", "owner":
"platform", "due": "2026-09-15", "regression_test": "evals/a
lias_pin.py"}]}
print(postmortem(rec))
try:
    postmortem({**rec, "actions": [{"what": "add alert"}]})
except ValueError as e:
    print("rejected:", e)
```

Interview Angle

Q: Your assistant leaked another tenant's document at 2 a.m. Walk through response. **A:** Declare top severity, name an incident commander, contain by disabling the affected retrieval path or cache and rolling back the last change, preserve traces and access logs, scope affected tenants from audit data, then notify per contract and legal advice. Fix the root cause (for example missing tenant in a key), add a canary leak test. Trade-off: containment may cause downtime. Check: leak probes pass, and notification timelines are met.

Q: Quality dropped 8 points overnight with no deploy. How do you investigate? **A:** Ask what changed outside your deploys: provider alias re-pointing, index refresh, upstream data, traffic mix, embedding or tool dependency, and config flags. Compare traces by bundle and model version, replay a fixed golden set against the current stack, and check judge drift. Trade-off: replays cost money and judge noise can mislead. Check: the replay on pinned inputs reproduces the drop and isolates the changed component.

Q: What makes a postmortem useful rather than ceremonial? **A:** Concrete causal analysis including the detection gap, action items with owners, dates and a regression test in the golden or red-team set, and follow-up review. Trade-off: thorough reviews take hours; scale depth to severity. Check: action-item completion rate, recurrence of the same cause, and MTTD trending down over quarters.